Runtime-Side Security Profile Generation for OCI Containers

Ivan Platonov[1] and Andrey Sadovykh[2]

[1] Innopolis University, Innopolis, Russia
i.platonov@innopolis.university
[2] Softeam, Paris, France
andrey.sadovykh@softeam.fr

Abstract. The widespread adoption of Open Container Initiative (OCI) containers, together with the increasing availability of automated offensive tooling, has made workload-specific container confinement a practical security requirement. Existing approaches fall into three broad categories, each with material limitations. Sandboxed runtimes such as gVisor reduce direct exposure to the host kernel by interposing a user-space kernel or microVM, but they introduce CPU, memory, and I/O overhead. Manual policy authoring can produce precise seccomp, AppArmor, and capability profiles, but it requires kernel-level expertise and per-workload validation effort that many engineering teams cannot sustain. Cluster-side recorders, including the Kubernetes Security Profiles Operator and Inspektor Gadget, automate parts of the workflow but require Kubernetes infrastructure, recorder storage, and an operational profile-pinning process.

This paper presents a security-oriented fork of `runc` in which profile generation is performed inside the runtime. During an explicit scanning invocation, the runtime relaxes confinement only in memory, executes the workload, records kernel interactions through eBPF-based syscall tracing, cgroup-scoped capability tracing, and AppArmor complain mode, and writes a workload-specific seccomp allow-list, AppArmor profile, and minimized capability set back into the OCI bundle. Subsequent executions enforce these artifacts as ordinary OCI inputs and require no recorder daemon, cluster-side component, or generated helper script; the tracing logic is packaged as hidden subcommands of the runtime binary, leaving the bundle portable. The evaluation framework compares the proposed runtime with unmodified `runc` and gVisor across runtime overhead, profile precision, and resistance to representative container-escape classes.

Keywords: OCI runtime · runc · seccomp · AppArmor · Linux capabilities · eBPF.

1 Introduction

Containers have become a standard deployment unit for cloud applications, but their isolation boundary remains fundamentally different from that of a virtual

machine. A container typically shares the host kernel with other workloads on the same node; consequently, kernel-facing attack surface directly affects tenant isolation and host integrity [12, 4]. The Open Container Initiative (OCI) standardizes the runtime contract, and `runc` remains the reference implementation beneath many higher-level container stacks. Its principal advantages are compatibility and low overhead; its principal limitation is that strong confinement depends on security policies supplied by the operator.

Linux already provides the mechanisms needed to narrow this boundary. Seccomp filters system calls, AppArmor constrains path-based filesystem access, and Linux capabilities decompose privileged root operations into smaller authorization units. The central challenge is therefore not the absence of enforcement mechanisms, but the cost of producing profiles that are restrictive enough to improve security while remaining permissive enough to preserve workload correctness. In practice, policy authoring requires kernel expertise, representative tests, and repeated debugging. Under delivery pressure, teams often retain default profiles or disable hardening when it interferes with deployment.

This paper investigates whether the runtime itself can reduce this cost. The proposed system is a fork of `runc` that adds a scanning mode to the normal runtime command path. During a scan, the runtime relaxes the bundle specification in memory, installs recording hooks, executes the workload, and writes generated artifacts back to the bundle. The scan is not an enforcement run; it is a learning phase intended for local testing or continuous integration. Later executions consume the generated seccomp, AppArmor, and capability policies as normal runtime inputs.

The contribution is threefold. First, the runtime exposes profile generation as a single-flag workflow rather than as a separate recorder stack. Second, it combines three confinement mechanisms in one pipeline: syscall recording through the OCI seccomp BPF hook, cgroup-wide capability tracing through BCC and `bpftool`, and AppArmor learning in complain mode. Third, it keeps scanner hooks inside the runtime binary, preserving OCI bundle portability and avoiding generated executable scripts in the bundle directory.

The remainder of the paper is organized as follows. Section 2 describes the runtime design and implementation. Section 3 defines the evaluation methodology. Section 4 analyzes the expected security and operational trade-offs. Section 5 positions the work against sandboxed runtimes, manual hardening, and cluster-side recorders. Section 6 concludes.

2 Implementation

2.1 Runtime Architecture

The implementation extends the normal `runc run` and `runc create` paths with an optional scanning branch. If the user does not pass `--security-scan`, the runtime executes the bundle using the ordinary OCI specification. If the flag is present, the runtime performs five steps: it relaxes confinement in memory,

installs hooks, runs the container, stops the tracers, and finalizes the generated profile set.

The relaxation step is deliberately confined to memory. The runtime does not widen the on-disk `config.json` before execution. Instead, it clears seccomp, clears the SELinux label, disables `NoNewPrivileges`, grants all known capabilities to the process, and replaces any AppArmor profile with a generated complain-mode profile. This design improves trace completeness: a policy that remains active during recording may suppress legitimate behavior and produce a profile that later breaks the application.

The generated artifacts are written under `generated/` in the bundle directory. Table 1 summarizes the stable output contract.

Table 1. Artifacts produced by a scan run.

Artifact	Purpose
`seccomp.json`	OCI seccomp allow-list produced by syscall tracing.
`capable-bpfcc.log`	Raw cgroup-wide capability trace consumed by finalization.
`apparmor.profile`	Generated AppArmor profile: loaded in complain mode during the scan, then updated from collected audit denials for enforcement.
`apparmor-load.log`	Loader diagnostics from `apparmor_parser`.
`capabilities-from-proc-status.txt`	Diagnostic snapshot of granted capabilities.
`spec.original.json`	Backup of the pre-scan specification.

2.2 Capability Narrowing

The capability pipeline is the most important part of finalization because it mutates the active `config.json`. The scanner runs BCC's `capable-bpfcc`, which observes `cap_capable` checks in the kernel [6]. A plain PID filter is insufficient because many workloads fork child processes after the container init has started. The implementation therefore uses `capable-bpfcc --cgroupmap`: it resolves the container cgroup, creates and pins a BPF map through `bpftool`, inserts the cgroup inode into that map, and starts the tracer against the pinned map.

Finalization parses the capability log, normalizes capability names to OCI `CAP_*` spelling, and replaces all five capability buckets: bounding, effective, permitted, inheritable, and ambient. The update is an evidence-based rebuild rather than a merge with previous or default capability sets. The final set starts empty and is extended only with capabilities observed by the scanner. If the trace contains no capability use, the resulting capability set is empty. This invariant prevents repeated scans from drifting toward broader defaults and enforces the

principle that a scan must not preserve a privilege merely because it appeared in an earlier configuration.

2.3 Seccomp and AppArmor Recording

System-call recording is delegated to `oci-seccomp-bpf-hook`, a maintained OCI hook used by container tooling for seccomp tracing [3]. The runtime sets the required tracing annotation and installs the hook as a prestart hook. The resulting `seccomp.json` remains an explicit artifact for later enforcement rather than being merged with any previous profile.

AppArmor recording follows a complain-mode workflow. The runtime writes a uniquely named profile, loads it with `apparmor_parser`, assigns it to the container process, and unloads it after the container stops. During post-stop finalization, it collects host audit denials for that profile, translates supported file and capability events into profile rules, and switches the generated profile to enforcement when rules have been collected. If AppArmor is unavailable, the profile file is still generated, but the runtime reports that the host cannot load it.

2.4 Self-Exec Hooks

The current implementation registers hidden runtime subcommands such as `scan-aa-load`, `scan-cap-trace-start`, and `scan-cap-trace-stop`. OCI hooks point back to the running `runc` binary, and parameters are passed through hook environment variables. Each subcommand drains hook state from standard input and writes diagnostics under `generated/`. The bundle therefore gains data artifacts only, not executable helper scripts.

3 Evaluation

The evaluation is designed to measure whether runtime-side profile generation can improve confinement while preserving the low overhead of a traditional OCI runtime. The comparison uses three configurations:

- default `runc`, representing the compatibility baseline;
- the proposed runtime with scan-generated enforcement profiles;
- gVisor, representing user-space-kernel and microVM sandboxing.

The workload set combines small services with application-like targets. Minimal workloads such as `busybox`, `nginx`, and `redis` provide low-noise performance baselines. A larger application workload, to be specified in the final benchmark matrix, provides a more realistic process tree and filesystem footprint. The matrix is intended to run on a fixed host configuration with cgroup v2, BCC tools, `bpftool`, AppArmor utilities, and `oci-seccomp-bpf-hook`.

The measured dimensions are listed in Table 2. Performance numbers should be reported as medians and interquartile ranges after a warm-up run. Security-effectiveness measurements should be reported per attack class rather than collapsed into a single score, because different configurations block attacks at different layers.

Table 2. Evaluation metrics.

Metric family	Measurements
Performance	Startup latency, steady-state CPU, memory footprint, I/O bandwidth, and TCP throughput.
Profile quality	Number of allowed syscalls, observed capabilities, AppArmor log entries, and generated artifact size.
Security effectiveness	Blocked attempts for syscall-bound escapes, capability-bound escapes, filesystem breakouts, and runtime-CVE-inspired attacks.
Operational cost	Required host dependencies, number of commands, generated files, and compatibility failures.

At the time of writing, the implementation and reporting schema are complete, while the final benchmark matrix remains pending. For this reason, the quantitative tables in this paper define the evaluation contract rather than completed empirical claims. The methodological objective is to compare the proposed runtime both with default `runc` and with stronger isolation systems, because the design targets an intermediate point: lower operational cost than cluster-side recording and lower runtime overhead than full sandboxing.

4 Analysis

4.1 Security Properties

The scanner improves security by reducing the default privileges available to a later enforcement run. Seccomp removes unused system calls from the kernel boundary; capability narrowing removes unused privileged operations; and AppArmor adds path-based visibility and confinement. These mechanisms are complementary: a capability profile can block privileged kernel checks, seccomp can remove entire syscall families, and AppArmor can restrict filesystem effects even when the corresponding system call remains allowed.

The strongest invariant is that finalization does not merge capability evidence with earlier broad defaults. A profile generator that unions current observations with previous capability sets would be operationally convenient but insecure by construction. The proposed runtime instead treats the scan as evidence: the output capability set is built from an empty set, and only observed capabilities survive. This makes the scan sensitive to test coverage, but it also makes the resulting privilege set auditable.

The cgroup-wide tracer closes a common blind spot in dynamic profiling. PID-based recording can miss helpers, workers, shell pipelines, and language runtime children. Filtering by cgroup better matches the container abstraction and is especially important for workloads that spawn subprocesses after initialization.

4.2 Limitations

The design inherits the limitations of dynamic tracing. A scan records only behavior that occurs during the scan window. Rare error paths, scheduled jobs, migration scripts, certificate renewal, and feature-flagged code can therefore be missed. The practical mitigation is to run scans under end-to-end tests that exercise representative behavior.

Root workloads are another limitation. Some capability checks are bypassed or under-reported for tasks running as UID 0, so scans of rootful workloads can produce incomplete capability evidence. The runtime warns about this case but does not rewrite the configured user, because doing so could change workload semantics.

Finally, AppArmor generation is automated for common audit-denial cases but still requires validation before production use. The runtime collects denials, inserts supported file and capability rules, and can switch the generated profile from complain mode to enforcement. Manual review remains necessary for unhandled AppArmor operations, missed execution paths, and decisions about whether learned filesystem access is intentionally required or merely an artifact of the test run.

4.3 Operational Trade-Off

The proposed runtime does not replace sandboxed runtimes for high-assurance multi-tenant isolation. gVisor reduces host-kernel exposure more fundamentally by interposing a user-space kernel or a microVM [13, 10]. The runtime-side scanner instead targets deployments in which default `runc` remains attractive for performance and compatibility, but workload-specific profile authoring is too expensive. Its value lies in reducing the setup cost from a recorder stack to a runtime flag.

5 Related Work

Prior work on container-runtime comparison consistently shows that `runc` is fast but weaker as an isolation boundary, whereas gVisor trades overhead for stronger isolation [13, 11, 10]. Those studies usually compare against default `runc`. This paper instead asks what a traditional runtime can achieve when workload-specific policies are generated automatically.

Automated seccomp generation has been studied directly by Lopes et al. [8], and static or hybrid approaches can recover execution paths that pure tracing

may miss [1]. The proposed runtime follows the dynamic tracing line but integrates tracing into the OCI runtime path and combines it with capabilities and AppArmor. Mattetti and Allouche [9] similarly argue for layered hardening across seccomp and mandatory access control.

Cluster-side recorders such as the Kubernetes Security Profiles Operator and tools such as Inspektor Gadget generate or assist with security profiles in Kubernetes environments [7, 5]. Tetragon uses eBPF for runtime observability and enforcement-oriented monitoring [2]. These systems are powerful, but they assume cluster infrastructure and operational expertise. The proposed runtime addresses the same policy-generation problem one layer lower, for developers who can execute a container but cannot operate a recorder stack.

6 Conclusion

This paper presented a `runc` fork that turns workload-specific profile generation into a runtime-side workflow. A single scanning flag records system calls, capability checks, and AppArmor-relevant behavior, then emits artifacts that can be used for later enforcement. The implementation keeps the scan explicit, keeps relaxation in memory, avoids generated executable hooks, and narrows capabilities according to observed behavior.

The main result is architectural rather than a claim of completed benchmarking: profile generation can reside inside the OCI runtime without requiring a Kubernetes recorder, a separate daemon, or manual policy authoring. The expected trade-off is a practical middle ground between default `runc` and heavier sandboxed runtimes. Future work should complete the benchmark matrix, add static augmentation for missed paths, emit Security Profiles Operator-compatible resources, and compare generated profiles directly against cluster-side recorders.

References

1. Canella, C., et al.: Automating seccomp filter generation for linux applications. In: Proceedings of ACM CCS 2021. pp. 2137–2152. ACM (2021)
2. Cilium / Isovalent: Tetragon: ebpf-based security observability and runtime enforcement. `https://github.com/cilium/tetragon` (2025), accessed 2026-04-19
3. containers organisation: oci-seccomp-bpf-hook: Oci prestart hook that traces syscalls and emits a seccomp profile. `https://github.com/containers/oci-seccomp-bpf-hook` (2024), accessed 2026-04-19
4. Flauzac, O., Gonzalez, C., Nolot, F.: A review of native container security for running applications. International Journal of Computer Science and Engineering **22**(3), 187–204 (2020)
5. Inspektor Gadget contributors: Inspektor gadget: collection of ebpf-based tools for inspecting and debugging kubernetes workloads. `https://github.com/inspektor-gadget/inspektor-gadget` (2025), accessed 2026-04-19
6. IO Visor / iovisor: Bcc: Tools for bpf-based linux io analysis, networking, monitoring, and more (`capable`, `capable-bpfcc`). `https://github.com/iovisor/bcc` (2025), accessed 2026-04-19

7. Kubernetes SIG Auth, kubernetes-sigs: Security profiles operator. `https://github.com/kubernetes-sigs/security-profiles-operator` (2025), accessed 2026-04-19
8. Lopes, N., Martins, R., Correia, M.E., Serrano, S., Nunes, F.: Container hardening through automated seccomp profiling. In: Proceedings of the 6th International Workshop on Container Technologies and Container Clouds. pp. 31–36. ACM (2020). https://doi.org/10.1145/3429885.3429966
9. Mattetti, M., Allouche, Y.: Automatic security hardening and protection of linux containers. Security and Privacy in Communication Systems (2020)
10. Viktorsson, W., Klein, C., Tordsson, J.: Security-performance trade-offs of kubernetes container runtimes. In: 2020 IEEE MASCOTS. pp. 1–4. IEEE (2020)
11. Volpert, S., Winkelhofer, S., Wesner, S., Domaschka, J.: An empirical analysis of common oci runtimes' performance isolation capabilities. In: Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering. pp. 60–70. ACM (2024). https://doi.org/10.1145/3629526.3645044
12. Wang, K., et al.: Characterizing and optimizing kernel resource isolation for containerized applications. Future Generation Computer Systems **141**, 234–247 (2023)
13. Wang, X., Johannesson, D.: Performance and isolation analysis of runc, gvisor and kata containers runtimes. Cluster Computing **25**, 2363–2380 (2022). https://doi.org/10.1007/s10586-021-03517-8